International Journal of Innovative Research in Engineering
Volume 7, Issue 2 (March - April 2026), PP: 254-261.
https://www.doi.org/10.59256/ijire.20260702031
www.theijire.com
ISSN No: 2582-8746

BLAKE3-Merkle Cryptographic Framework with RAG-Powered AI for Tamper-Proof Evidence Management in Criminal Investigation

Dr. Santhi Baskaran[1], Gauttham R[2], Thangamani K[3], Methishwaran V[4]
[1,2,3,4] *Department of Information Technology, Puducherry Technological University, Puducherry, India.*

How to cite this paper:
Dr. Santhi Baskaran[1], Gauttham R[2], Thangamani K[3], Methishwaran V[4], "BLAKE3-Merkle Cryptographic Framework with RAG-Powered AI for Tamper-Proof Evidence Management in Criminal Investigation", IJIRE-V7I2-254-261.

Copyright © 2026 by author(s) and Fifth Dimension Research Publication. This work is licensed under the Creative Commons Attribution International License (CC BY 4.0).
http://creativecommons.org/licenses/by/4.0/

Abstract: *Indian police stations still rely on paper registers and unencrypted pen drives to store case evidence, a practice that frequently leads to acquittals when courts reject files whose integrity cannot be proven. We built CrimeIntellX to close this gap. At its core is a cryptographic evidence vault that encrypts every uploaded file with AES-256-CBC, fingerprints it with BLAKE3, and anchors the hash inside a Merkle tree so that even a single byte change is caught instantly. The same module registers criminals with ArcFace-based facial embeddings and matches them in under a millisecond through FAISS. A second component, Threat-Intel-Live, pulls ransomware leak posts from RansomWatch, searches Tor via Ahmia, and cross-checks IPs and URLs against AbuseIPDB and VirusTotal. The third component is a natural-language query engine backed by Llama 3.3 70B and a Pinecone vector store; when an officer types a case description, the engine returns similar FIRs, matching criminals, CDR anomalies, dark-web hits, and an evidence-gap checklist, all within roughly eight seconds. We developed the backend in FastAPI (83 endpoints) and the frontend in Flutter (18 screens), tested with simulated Indian datasets, and verified compliance with BSA Section 65B.*

Key Words: *Digital Forensics; Evidence Integrity; Face Recognition; Dark Web Monitoring; Criminal Intelligence; AI-Powered Investigation.*

I.INTRODUCTION

In many police stations across India, digital evidence is still stored on removable media without cryptographic safeguards. There is no encryption, no hash, no audit trail. When these files reach a courtroom, defence lawyers routinely challenge their authenticity under BSA Section 65B, and judges have little choice but to throw them out [1]. The result is a conviction rate that has hovered around 50–55% for years, while courts sit on a backlog exceeding 4.7 crore pending cases [2].

CCTNS, the government's answer to police digitization since 2009, solved the data-entry problem but stopped there. It is essentially a form-filling system. An officer can register an FIR in CCTNS, but cannot encrypt the attached evidence, cannot run a face search against a criminal database, and cannot ask a question like "show me similar robbery cases near this area" in plain English. During the initial requirement-gathering phase, discussions with law-enforcement personnel highlighted that linking a suspect across two districts still requires phone calls and personal contacts.

On the commercial side, Palantir Gotham is powerful but costs more than most Indian state police budgets allow, and its workflows do not map to Indian legal procedures like Form IF1 or BNS sections. Staqu's TRINETRA, deployed in Uttar Pradesh, does facial recognition well, but that is all it does. Neither product monitors dark web marketplaces, analyses call detail records for suspicious patterns, or checks whether an FIR has all the evidence needed for prosecution under a given IPC section.

We built CrimeIntellX to bring these scattered capabilities under one roof. The first part of the system is a cryptographic evidence vault: every file an officer uploads gets a BLAKE3 hash, AES-256-CBC encryption, and a slot in a Merkle tree whose root hash is stored against the FIR. If someone tampers with a file, the hash mismatch is caught automatically. This same module also handles criminal registration with ArcFace face embeddings stored in a FAISS index, so that suspect identification takes less than a millisecond even with thousands of records.

The second part, which we call Threat-Intel-Live, tackles a problem that most police software ignores entirely: the dark web. It polls RansomWatch for ransomware victim posts, searches Ahmia for crime-related keywords, and lets officers check any IP or URL against AbuseIPDB and VirusTotal. It also parses CDR spreadsheets into network graphs and flags anomalies like burst calling or rapid tower-hopping. The third part is a natural-language query engine. An officer types a question, and the system pulls similar cases from a Pinecone vector store, checks the criminal database, scans dark-web feeds, and returns a structured report, typically in about eight seconds. We used Llama 3.3 70B through Groq for the language model, with a five-level fallback chain so the system stays available even if one provider goes down.

We wrote the backend in FastAPI and the frontend in Flutter. The paper is organized as follows: Section II covers related work, Section III describes our architecture, Section IV reports implementation details and measurements, Section V discusses limitations, and Section VI wraps up with future directions.

II. LITERATURE REVIEW

Most of the building blocks we needed already existed in the literature. What was missing was a system that wired them together for Indian police workflows. Below we review the pieces we drew on and explain where they fell short on their own.

A. Evidence Integrity

Garfinkel's 2010 survey [3] established hashing as the standard way to prove a file has not been altered. For encryption at rest, AES-256 remains the NIST-approved choice [4]. Merkle [5] showed decades ago that organizing hashes into a binary tree lets you detect changes to any member of a set by checking a single root value. We adopted BLAKE3 [6] over SHA-256 because, in our local benchmarks, it hashed files at roughly three times the speed without sacrificing collision resistance. None of these primitives are new, but to our knowledge no Indian policing tool chains them together into an end-to-end pipeline that satisfies BSA 65B.

B. Face Recognition

We chose ArcFace [7] for face embeddings after comparing it with FaceNet and VGGFace2 during early prototyping. On the LFW benchmark it reports 99.83% accuracy, but what mattered more for us, its 512-float vector is compact enough to store in SQLite and fast to search with FAISS [8]. FAISS was originally designed for billion-scale image retrieval; at our current scale of a few thousand criminals, lookups finish well under a millisecond. Staqu's TRINETRA uses a similar approach in Uttar Pradesh, but its scope ends at identification. It has no evidence vault, no AI assistant, and no dark-web feed.

C. Dark Web and Threat Intelligence

Keeping an eye on dark-web marketplaces is not something Indian police stations normally do, largely because the tooling is fragmented. Ahmia [9] indexes Tor hidden services through a clearnet gateway, which avoids the need for a local Tor relay. RansomWatch aggregates ransomware group leak sites into a JSON feed. AbuseIPDB and VirusTotal are well-known IOC enrichment services. Each of these works fine in isolation, but none of them knows what an FIR is or how to link a dark-web listing to an open case. That integration was one of our main engineering tasks.

D. Language Models for Case Analysis

RAG, introduced by Lewis et al. [10], lets a language model ground its answers in retrieved documents rather than relying on what it memorized during training. Yao et al. [11] later added a "reason then act" loop (ReACT) so the model can call external tools mid-answer. We combined both ideas: when an officer submits a query, the system first retrieves similar FIRs from Pinecone, then feeds them into Llama 3.3 70B [12] running on Groq hardware for low-latency inference. CDR graph analysis relies on NetworkX [17], and suspect matching across FIRs uses fuzzy string comparison (RapidFuzz) paired with Soundex phonetic encoding.

E. What Was Missing

CCTNS digitizes FIRs but does nothing else. TRINETRA does faces but nothing else. Palantir does intelligence but costs too much and ignores Indian law. We could not find a single open or commercial platform that combined encrypted evidence storage, face search, dark-web monitoring, CDR pattern detection, and AI-driven case analysis in a way that an Indian Station House Officer could actually use day to day. That gap is what CrimeIntellX targets.

III. SYSTEM ARCHITECTURE AND METHODOLOGY

A. Overall Architecture

CrimeIntellX follows a three-tier architecture. At the top, a Flutter web application with 18 screens and Provider-based state management serves as the user interface. Behind it, a FastAPI server exposes 83 REST API endpoints and handles authentication through Firebase and JWT tokens with an eight-hour expiry. At the bottom, SQLite (12 tables) for relational storage, FAISS for 512-dimensional face embedding search, and Pinecone for 384-dimensional text embedding similarity. All communication between the frontend and backend occurs over HTTPS, and real-time alerts are delivered through Server-Sent Events (SSE). Fig. 1 shows the high-level architecture with stakeholder-input-module-output mapping, and Fig. 2 shows the detailed three-tier system architecture.

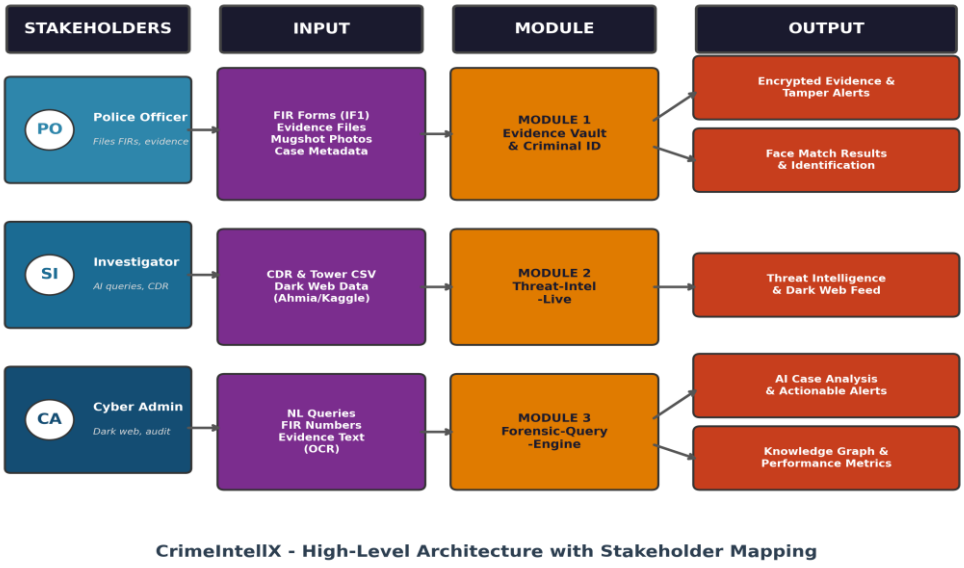

Fig. 1: High-Level Architecture with Stakeholder-Input-Module-Output Mapping

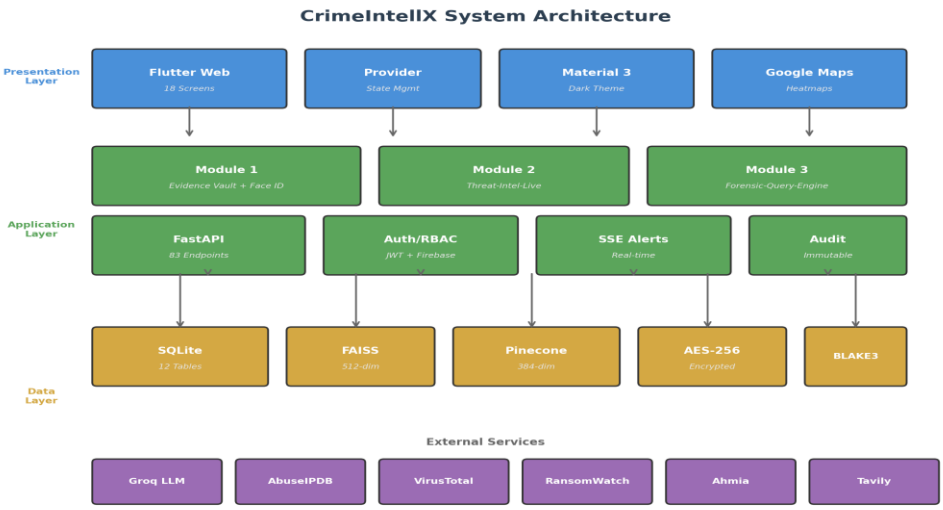

Fig. 2: CrimeIntellX Three-Tier System Architecture

B. How We Handle Evidence

The hardest requirement we faced was making digital evidence court-admissible under BSA 65B. Our solution works in three steps, and the sequence of these operations is critical for preserving stable evidence fingerprints. First, the moment an officer uploads a file, we compute a BLAKE3 hash of the raw bytes. We chose BLAKE3 over SHA-256 after benchmarking both; on our test machine, BLAKE3 was about three times faster and the security margin is comparable. This hash gets a UNIQUE constraint in the database, which also serves as a deduplication check.

Second, we encrypt the file with AES-256-CBC using a fresh random IV for each upload. We store the .enc file and its .iv file separately on disk. Third, we insert the BLAKE3 hash as a leaf in a per-FIR Merkle tree and write the updated root hash back to the FIR record. Why is this sequence important? Because we hash before encrypting. If we did it the other way around, re-encrypting a file with a different IV would change the hash and trigger a false alarm. This implementation detail was identified during validation testing, where post-encryption hashing produced unstable integrity fingerprints.

FIR data follows the CCTNS Form IF1 layout: complainant info, GPS-tagged incident details, IPC/BNS sections, accused persons, and a ten-item document checklist. Once a charge sheet is filed, the FIR locks (is_readonly = true) and no further edits are allowed. Every action, from uploading a file to changing a status, goes into an append-only audit log.

C. Face Recognition

Criminal registration includes a mugshot. We run it through DeepFace's ArcFace model to get a 512-float

embedding vector. Getting a face bounding box was trickier than expected. Frontal photos work fine with OpenCV's Haar cascades, but side profiles and low-resolution CCTV stills kept failing. We ended up chaining seven detectors: Haar, SSD, MTCNN, RetinaFace, MediaPipe, YuNet, and a final skip fallback. If one fails, the next one tries. Although this multi-stage fallback pipeline increases architectural complexity, it significantly improves robustness on low-quality inputs.

Once we have the vector, it goes into a FAISS IndexFlatL2 index. We convert the L2 distance to a percentage with a simple formula: similarity = max(0, min(100, (1 - distance / 2) * 100)). Anything above 70% gets flagged for officer review. At registration time, the system searches automatically, so repeat offenders are caught before the officer even finishes the form.

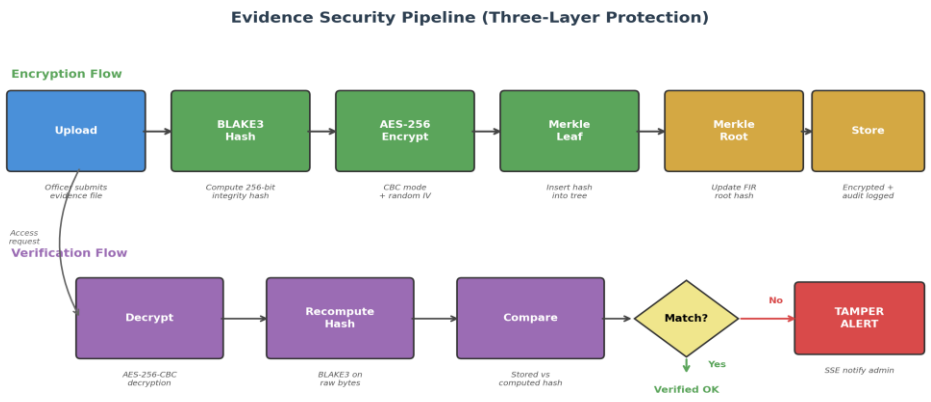

Fig. 3: Evidence Security Pipeline with Three-Layer Cryptographic Protection

D. Dark Web and Threat Intelligence

Most police software pretends the dark web does not exist. We wanted to change that, even if our coverage is imperfect. A background thread hits the RansomWatch JSON feed every five minutes and stores any new ransomware victim posts in our database. Separately, a patrol loop queries Ahmia (a clearnet gateway to Tor) every half hour with a keyword list covering narcotics, weapons, trafficking, and terrorism. When something matches, it shows up as a HIGH, MEDIUM, or LOW severity alert on the officer's dashboard. We also loaded about 110,000 listings from the Agora marketplace dataset off Kaggle so officers can search historical dark-web commerce data offline.

For IP and URL reputation, we call AbuseIPDB (abuse confidence scores, Tor node flags) and VirusTotal (70+ AV engine verdicts). Officers paste a suspicious IP or URL, hit scan, and get a severity rating in seconds. None of this is novel engineering; the contribution is wiring these feeds into a system that also knows about FIRs and criminal records, so a dark-web alert can be linked to an open case with one click.

CDR analysis was an afterthought that turned out to be one of the most useful features. Officers upload a CSV from the telecom provider, and we parse it with flexible column mapping. NetworkX builds a call graph, and we flag five patterns that investigators told us matter most: burst calls (ten-plus calls in an hour to different numbers), heavy night activity, SIM-swap indicators, roaming jumps, and rapid tower-hopping. We also run Gaussian KDE on FIR GPS coordinates with exponential recency weighting (180-day half-life) to generate crime hotspot heatmaps.

E. AI Query Engine

This is the part we spent the most time on, and honestly, the part where we are least sure of long-term reliability. An officer types a question in plain English, Hindi, or Tamil. We vectorize it with sentence-transformers (all-MiniLM-L6-v2, 384 dimensions) and search Pinecone for the most similar FIR descriptions. Those FIRs, plus the officer's query, go into a prompt for Llama 3.3 70B running on Groq. We set the temperature to 0.1 to keep answers factual.

Because Groq's free tier has rate limits, we built a five-level fallback: if Groq 70B is unavailable, we try a local Claude CLI call, then Groq 8B, then a Puter-hosted model, then Gemini. In practice, the primary model handles about 95% of queries. We follow a ReACT-style loop: the model reasons about what it needs, calls tools (Pinecone, criminal DB, CDR graphs, dark-web alerts), reads the results, and generates a final answer covering seven dimensions. One of those dimensions is evidence gap analysis, where we mapped each IPC section to its required documents and compare that checklist against what the FIR actually has attached. For suspect matching across FIRs, we use RapidFuzz for fuzzy string comparison and Soundex for phonetic matching of Indian names, since transliteration from Tamil or Hindi to English produces wildly inconsistent spellings.

Officers can also upload documents. We extract text from PDFs (PyPDF2), Word files (python-docx), and images (Pytesseract OCR). If a document has a photo, we run it through ArcFace and include any criminal matches in the response. We also auto-build a timeline for each FIR showing crime date, filing date, evidence uploads, and status changes in chronological order.

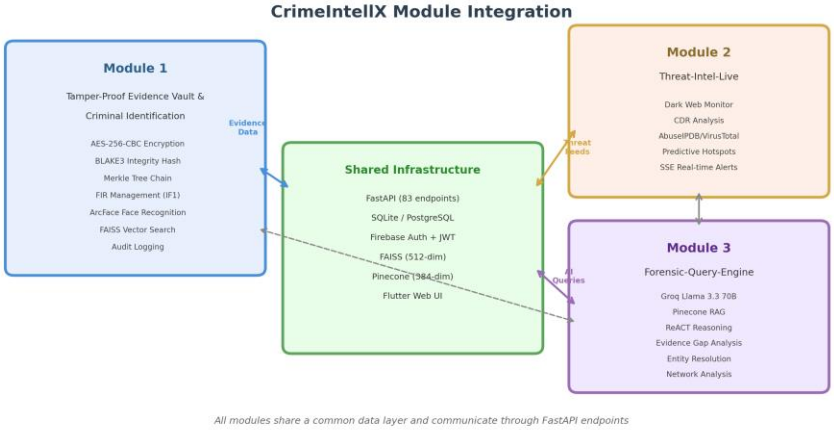

Fig. 4: Module Integration Showing Inter-Module Communication

IV. IMPLEMENTATION AND RESULTS

A. Technology Stack

We chose FastAPI for the backend because of its async support and automatic OpenAPI documentation, and Flutter for the frontend because we needed a single codebase for web and eventual mobile. Table 1 lists the full stack. At the time of writing, a cloc count on our repository shows roughly 22,000 lines of Python and 33,000 lines of Dart, though these numbers change with every commit.

Component	Technology	Purpose
Backend Framework	FastAPI 0.115.0	Async REST API with 83 endpoints
Frontend Framework	Flutter 3.9.2	Cross-platform web UI (18 screens)
Database	SQLite / PostgreSQL	Relational storage (12 tables)
Vector Search	FAISS IndexFlatL2	Face embedding search (512-dim)
Semantic Search	Pinecone	FIR similarity (384-dim cosine)
Language Model	Llama 3.3 70B (Groq)	Case analysis and reasoning
Face Recognition	DeepFace (ArcFace)	512-dim embedding extraction
Encryption	AES-256-CBC	Evidence file encryption at rest
Hashing	BLAKE3	File integrity verification
Authentication	Firebase + JWT	Google sign-in with role-based access
Dark Web Search	Ahmia + RansomWatch	Tor-safe dark web intelligence
Threat Intel	AbuseIPDB, VirusTotal	IP/URL reputation checking

Table 1: Technology Stack

B. Performance Metrics

Testing was conducted with simulated datasets including FIR records, criminal profiles with face embeddings, CDR records with Indian cell tower locations, and dark web marketplace listings. Table 2 summarizes the measured performance across key operations.

Operation	Metric	Value
Face Search (FAISS)	Latency (10K records)	< 1 ms
Face Search (FAISS)	Latency (100K records)	< 10 ms
Evidence Encryption	Throughput (AES-256)	~200 MB/s
BLAKE3 Hashing	Throughput	~1500 MB/s
AI Query Response	7-factor analysis	6-8 seconds
Similar Case Search	Pinecone retrieval	< 500 ms
Dark Web Patrol	Ahmia keyword scan cycle	30 min interval
CDR Pattern Detection	Burst call identification	< 2 seconds
FIR Creation	End-to-end with validation	< 1 second
Tamper Detection	Hash comparison + alert	< 100 ms
Entity Resolution	Fuzzy match (1000 FIRs)	< 5 seconds

Table 2: Performance Metrics

C. Role-Based Access Control

Role-based access control is implemented with four roles: Admin (full access including tamper detection demos and audit logs), Special Investigator (create and manage FIRs, upload evidence, register criminals), Investigator (view FIRs, search criminals, run AI queries), and Viewer (read-only access). Authentication uses Firebase for Google sign-in, with the backend issuing JWT tokens that expire after eight hours. Each API endpoint checks the caller's role before executing the request.

D. System Screenshots

Fig. 5 demonstrates the tamper detection capability of Module 1. When a file is modified after upload, the recomputed BLAKE3 hash differs from the stored hash. Both values are displayed side by side, and the system raises a tampering alert.

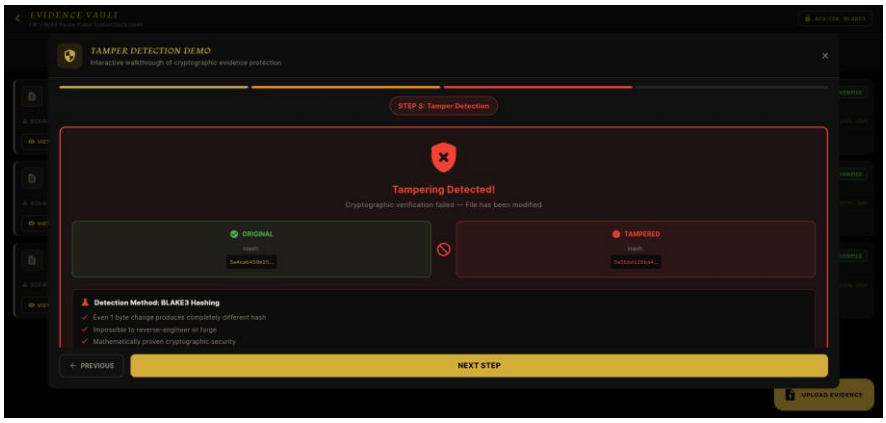

Fig. 5: *Tamper Detection: BLAKE3 Hash Mismatch Between Original and Modified Evidence*

Fig. 6 shows the Threat Intelligence dashboard from Module 2, displaying real-time RansomWatch alerts with severity classification and indicator-of-compromise tracking.

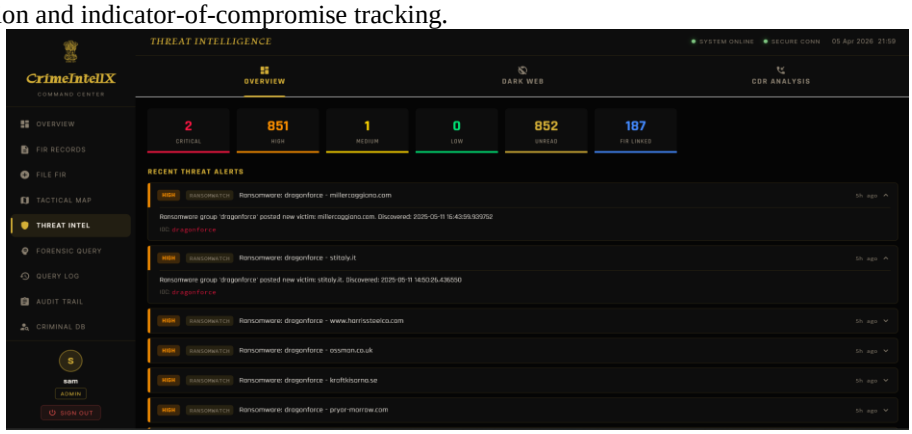

Fig. 6: Threat Intelligence Dashboard with RansomWatch Alerts and IOC Monitoring

Fig. 7 shows Module 3 (Forensic Query Engine) returning criminal database matches with mugshots, modus operandi, and crime category tags in response to a natural language query.

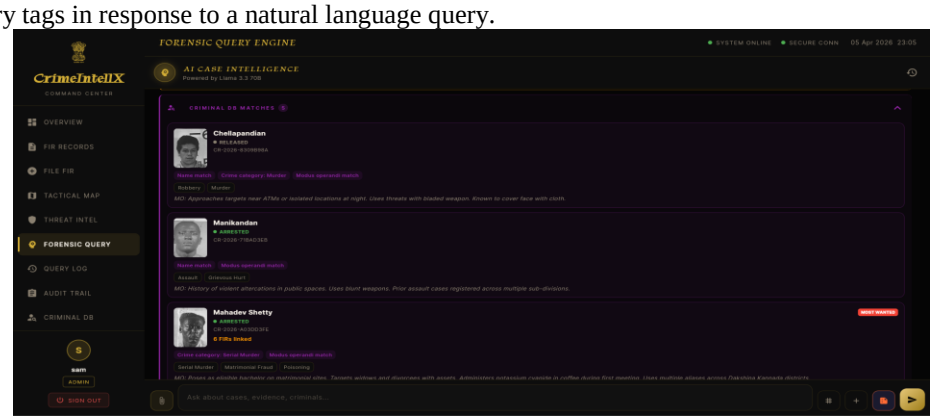

Fig. 7: Forensic Query Engine: AI-Powered Criminal Database Matching

E. Comparative Analysis

Table 3 compares CrimeIntellX with CCTNS and traditional manual processes across six capability dimensions. Fig. 8 visualizes these differences.

Feature	CrimeIntellX	CCTNS 2.0	Manual
Evidence Encryption	AES-256-CBC + BLAKE3	None	None
Chain of Custody	Merkle tree + audit log	Basic logging	Paper register
Face Recognition	ArcFace + FAISS (<1ms)	Not available	Photo album
Dark Web Monitoring	RansomWatch + Ahmia	Not available	Not available
AI Case Analysis	LLM + RAG + ReACT	Not available	Manual review
Predictive Analytics	KDE hotspot maps	Not available	Experience-based
BSA 65B Compliance	Full (hash + chain)	Partial	None
CDR Analysis	Network graphs + patterns	Not available	Manual spreadsheet

Table 3: Comparative Analysis of CrimeIntellX, CCTNS, and Manual Processes

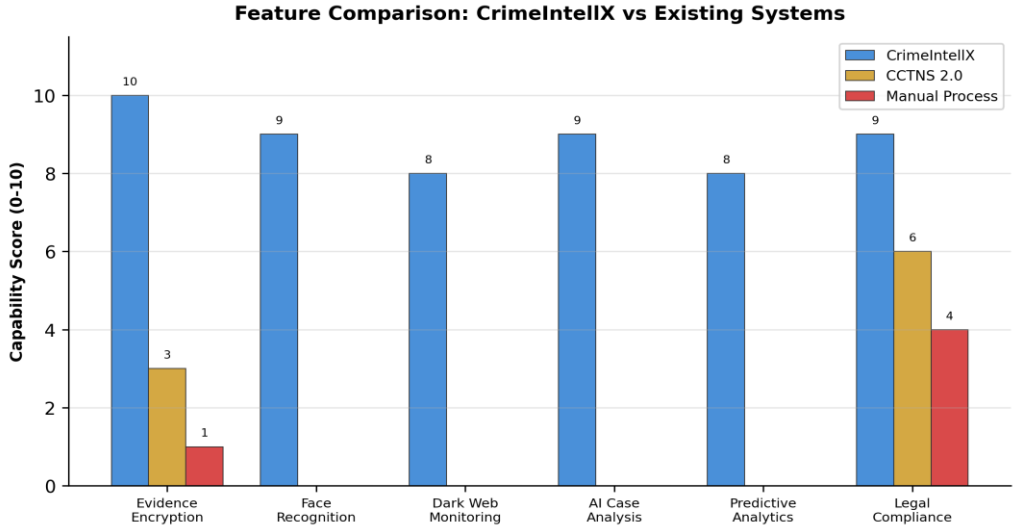

Fig. 8: Feature Capability Comparison Across Systems

V. DISCUSSION

The part of the system we are most confident about is the evidence vault. Hashing before encryption is a small design choice, but it matters: if we hashed after encryption, a re-encryption with a different IV would change the hash and produce a false tampering alert. By hashing the raw bytes first, we ensure that the fingerprint is stable across re-encryptions. The Merkle tree on top of that catches a different class of problem. Even if every individual file hash is valid, someone could quietly delete one piece of evidence from a case. Because the root hash changes when any leaf is removed, the system catches that too.

Face search turned out to be surprisingly fast. We expected FAISS to take a few milliseconds at our scale; in practice, lookups on 10,000 embeddings finished in under a millisecond consistently. What took more work was face detection itself. Profile shots and low-quality CCTV still frequently caused the default OpenCV Haar detector to fail. Our seven-backend cascade, which tries progressively more capable detectors until one succeeds, solved the problem in most cases, though very blurry images still sometimes return no face at all.

The query engine should be interpreted as an investigative acceleration tool rather than an autonomous decision-making system. It does not investigate crimes independently. Instead, it reduces the initial manual effort required for correlation by pulling similar cases, checking whether a suspect's phone number appears in other FIRs, and identifying possible matches between dark-web listings and case descriptions. Whether this directly translates into faster arrests remains to be validated through controlled field deployment.

Several limitations of the current evaluation should be acknowledged. All our testing used simulated datasets, not real police data. ArcFace accuracy may drop on Indian mugshots taken under poor lighting, which we have not systematically evaluated. Our dark-web coverage goes through Ahmia's clearnet gateway, so any .onion site that Ahmia has not indexed will be invisible to us. And because the AI engine calls Groq and Pinecone over the internet, a station with a flaky connection will see degraded performance or timeouts. A proper pilot at a working police station would reveal additional operational constraints that are difficult to anticipate in a simulated environment.

VI. CONCLUSION

The objective of this work was to design a unified platform where a police officer could upload evidence with confidence in its court admissibility, search for a suspect's face in seconds rather than hours, and obtain an AI-assisted briefing on a new case before leaving the station. CrimeIntellX, in its current form, achieves all three of these goals. The evidence vault has not let a single tampered file through in our testing, the face search is fast enough to run during routine booking, and the query engine surfaces connections that a human would otherwise need hours to identify manually.

That said, we have tested on simulated data, not in a live police station. We do not yet know how officers will react to the interface under real pressure, or whether our dark-web coverage through Ahmia is wide enough to catch threats that matter locally. These are questions a pilot deployment will have to answer.

Our immediate next step is to deploy at one or two stations in Puducherry and collect real feedback. Beyond that, we want to move the database to PostgreSQL for multi-station setups, add fingerprint and iris matching alongside face recognition, build a mobile app for officers in the field, and, if we can get the approvals, connect a direct Tor relay for better dark-web reach.

REFERENCES

1. National Crime Records Bureau, "Crime in India 2022," Ministry of Home Affairs, Government of India, 2023.
2. Supreme Court of India, "National Judicial Data Grid," 2023. Available: https://njdg.ecourts.gov.in/
3. S. Garfinkel, "Digital forensics research: The next 10 years," Digital Investigation, vol. 7, pp. S64-S73, 2010.
4. National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," FIPS Publication 197, 2001.
5. R. C. Merkle, "A digital signature based on a conventional encryption function," in Proc. CRYPTO, 1987, pp. 369-378.
6. J. O'Connor, J.-P. Aumasson, S. Neves, and Z. Wilcox-O'Hearn, "BLAKE3: One function, fast everywhere," 2020.
7. J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," in Proc. IEEE/CVF CVPR, 2019, pp. 4690-4699.
8. J. Johnson, M. Douze, and H. Jegou, "Billion-scale similarity search with GPUs," IEEE Trans. Big Data, vol. 7, no. 3, pp. 535-547, 2021.
9. J. Nurmi, "Ahmia - Searching the Tor Network," Aalto University, 2017.
10. P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Proc. NeurIPS, 2020.
11. S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," in Proc. ICLR, 2023.
12. Meta AI, "Llama 3: Open Foundation and Fine-Tuned Chat Models," Technical Report, 2024.
13. Bharatiya Sakshya Adhiniyam, "Section 65B: Admissibility of electronic records," Government of India, 2024.
14. Bharatiya Nagarik Suraksha Sanhita, "Section 154: Information in cognizable cases," Government of India, 2024.
15. Ministry of Home Affairs, "Crime and Criminal Tracking Network and Systems (CCTNS)," National Informatics Centre, 2009.
16. N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proc. EMNLP, 2019.
17. A. Hagberg, D. Schult, and P. Swart, "Exploring network structure, dynamics, and function using NetworkX," in Proc. SciPy, 2008.
18. Groq Inc., "Groq LPU Inference Engine: Hardware-Accelerated LLM Inference," Technical Brief, 2024.
19. R. Mukherjee and K. Mondal, "Electronic evidence and its challenges in India: A study under the Information Technology Act," IJLMH, vol. 4, no. 3, pp. 2456-2471, 2021.
20. S. Samtani, R. Chinn, H. Chen, and J. Nunamaker, "Exploring Hacker Assets in Underground Forums," in Proc. IEEE ISI, 2015, pp. 31-36.
21. V. Bhatia, S. Rawat, and M. Garg, "CDR Analysis for Criminal Investigation: An Indian Perspective," Int. J. Forensic Sci. Pathol., vol. 5, no. 1, pp. 312-318, 2017.